NEUROMORPHIC COMPUTING

Spiking Neural Network Motion Direction Classifier

Supporting Documentation

Project Type	**Simulation Tool**
Neuromorphic Computing Simulation	Brian2 (Python)
Task	**Test Accuracy**
Binary motion direction classification	100% | Latency: 14.7 ms mean

Author: Iyed Douggaz

Class: 2DAD

School: ENSTAB

0 Contents

1 Project Overview **2**
1.1 Objectives . . . 2
1.2 Tools & Libraries . . . 2

2 System Architecture **2**
2.1 Pipeline Block Diagram . . . 3
2.2 Component Table . . . 4
2.3 Synapse Architecture . . . 4

3 Event-Based Sensor (Synthetic DVS) **4**
3.1 Real vs. Synthetic DVS . . . 5
3.2 Event Generation Process . . . 5
3.3 Spike Preprocessing . . . 5

4 Spiking Neural Network **5**
4.1 Neuron Model — Leaky Integrate-and-Fire (LIF) . . . 5
4.2 Network Topology . . . 6
4.3 Hyperparameter Justification . . . 6

5 Readout Classifier & Latency **7**
5.1 Feature Extraction . . . 7
5.2 Classifier Pipeline . . . 7
5.3 Latency Measurement . . . 7

6 Results **8**
6.1 Summary Table . . . 8
6.2 Classification Performance . . . 8
6.3 Latency Analysis . . . 8
6.4 Simulation Outputs . . . 9

7 Limitations & Future Work **9**
7.1 Current Limitations . . . 9
7.2 Suggested Next Steps . . . 9

8 Glossary **10**

1 Project Overview

This project implements a complete neuromorphic computing pipeline for binary motion direction classification. A synthetic Dynamic Vision Sensor (DVS) generates event-based spike data representing a moving vertical bar. These events drive a recurrent Leaky Integrate-and-Fire (LIF) spiking neural network simulated in Brian2. A logistic readout classifier then decodes the network's hidden spike activity to determine whether the bar moved rightward or leftward.

Key result: 100% classification accuracy with a mean decision latency of 14.7 ms on a 400 ms trial — the network decides in the first 4% of the stimulus window.

This demonstrates the core neuromorphic advantage: ultra-low-latency inference driven by sparse, asynchronous spike events rather than dense frame-by-frame computation.

1.1 Objectives

- Simulate biologically plausible event-based vision using a synthetic DVS sensor
- Build and tune a recurrent LIF spiking neural network in Brian2
- Demonstrate time-driven response through N-th spike latency measurement
- Achieve robust classification of motion direction (right vs. left)
- Produce reproducible simulation outputs, plots, and analysis

1.2 Tools & Libraries

Tool / Library	Version	Role
Python	3.10+	Primary programming language
Brian2	2.x	Spiking neural network simulator
NumPy	1.x	Numerical computation, spike processing
scikit-learn	1.x	Logistic readout, MaxAbsScaler, pipeline
Matplotlib	3.x	Raster plots, confusion matrices, histograms
tqdm	4.x	Progress bars for long simulation loops
Pandas	2.x	Per-trial metrics export to CSV

2 System Architecture

The pipeline has seven sequential stages, from raw event generation through to latency measurement. Each stage is implemented as a standalone Python function, making the pipeline modular and easy to extend.

2.1 Pipeline Block Diagram

Stage & Function / Module	**Output / Description**
1. DVS Event Generator `generate_moving_bar_events()`	Synthetic ON/OFF events (x, y, t, pol)
2. Spike Pre-processor `events_to_spikegenerator_args()`	Sorted + deduplicated → Brian2-safe (indices, times)
3. SpikeGeneratorGroup Brian2 built-in	Input neuron layer — 2048 neurons (32×32×2)
4. Recurrent LIF Network NeuronGroup + 3× Synapses	128 hidden neurons — feedforward excitation + lateral inhibition
5. Time-Binned Feature Extract `extract_binned_hidden_counts()`	128 × 16 time bins → 2048-dim feature vector
6. Logistic Readout Classifier MaxAbsScaler + LogisticRegression	Binary decision: right (0) vs. left (1)
7. N-th Spike Latency `compute_latency_nth_spike()`	Time of 10th hidden population spike → decision latency

2.2 Component Table

#	Component	Code / Module	Description
1	DVS Event Generator	`generate_moving_bar_events()`	Synthetic ON/OFF spike events (x, y, t, pol)
2	Spike Pre-processor	`events_to_spikegenerator_args()`	Sorted, deduplicated Brian2-safe (indices, times)
3	Spike-Generator-Group	Brian2 built-in	Injects input spikes into simulation
4	LIF Hidden Population	NeuronGroup + 3× Synapses	$dv/dt = -v/\tau$; fires when $v > 1$
5	Time-Binned Feature	`extract_binned_hidden_counts()`	128 neurons × 16 bins → 2048-dim vector
6	Logistic Readout	MaxAbsScaler + LogisticRegression	Binary classifier: right vs. left
7	Latency Measurement	`compute_latency_nth_spike()`	Time of 10th hidden population spike

2.3 Synapse Architecture

The LIF hidden population receives three distinct synapse types. The combination of feedforward excitation, recurrent excitation, and lateral inhibition creates competitive dynamics that sharpen neural selectivity and prevent firing-rate saturation.

Name	Source → Target	Effect	Wt. / Dens.	Role
S — Feedforward Exc.	Input (G_{in}) → Hidden (G)	$v \mathrel{+}= w$	0.08 / 6%	Drives initial response
R — Recurrent Exc.	Hidden (G) → Hidden (G)	$v \mathrel{+}= w$	0.03 / 2%	Sustains activity
I — Lateral Inh.	Hidden (G) → Hidden (G)	$v \mathrel{-}= w$	0.05 / 5%	Prevents saturation

3 Event-Based Sensor (Synthetic DVS)

A Dynamic Vision Sensor (DVS) differs fundamentally from a conventional camera. Instead of capturing full frames at a fixed frame rate, each pixel operates independently and fires an event only when it detects a change in log-luminance. This produces a sparse, asynchronous stream of $(x, y, \text{timestamp}, \text{polarity})$ events with microsecond temporal resolution.

3.1 Real vs. Synthetic DVS

Property	Real DVS (e.g. DAVIS346)	Our Synthetic Generator
Output format	$(x, y, t, \text{polarity})$	$(x, y, t, \text{polarity})$ — identical
Polarity	ON (+1) and OFF (−1)	ON (0) and OFF (1)
Temporal resolution	∼1 µs	Controlled by fps + Gaussian jitter ($\sigma = 4\,\text{ms}$)
Noise model	Shot noise, hot pixels	Gaussian temporal jitter on event timestamps
Speed variation	Object-dependent	Gaussian speed variation ($\sigma = 12\,\text{px/s}$)
Resolution	346×260 typical	32×32 (fast simulation)

3.2 Event Generation Process

- A vertical bar of width 4 px sweeps horizontally across the 32×32 grid at ∼100 px/s
- ON events (polarity = 0) are emitted for every pixel currently covered by the bar
- OFF events (polarity = 1) are emitted for pixels the bar has just left
- Gaussian jitter ($\sigma = 4\,\text{ms}$) is added to each event timestamp to simulate sensor noise
- Speed is randomised per trial ($\sigma = 12\,\text{px/s}$) to ensure classifier generalisation
- Total input neurons: $32 \times 32 \times 2\,\text{polarities} = 2048$

3.3 Spike Preprocessing

Before entering Brian2, events are processed by `events_to_spikegenerator_args()`:

- **Neuron index mapping:** $\text{idx} = y \times W + x + \text{polarity} \times (W \times H)$
- Events sorted by timestamp (required by `SpikeGeneratorGroup`)
- Per-neuron minimum inter-spike interval enforced (0.2 ms) to prevent Brian2 timestep collision errors
- Spike times clipped to simulation window [0, sim_duration_s)

4 Spiking Neural Network

4.1 Neuron Model — Leaky Integrate-and-Fire (LIF)

Each hidden neuron follows the standard LIF dynamics:

$$\frac{dv}{dt} = -\frac{v}{\tau}$$

- **Threshold:** $v > 1 \;\Rightarrow\;$ spike emitted
- **Reset:** $v = 0$ (hard reset after spike)

- **Time constant:** $\tau = 20\,\text{ms}$
- **Integration method:** Exact (analytical solution, no numerical error)

The membrane potential v decays exponentially toward zero in the absence of input. Each presynaptic spike causes an instantaneous jump in v. When v crosses the threshold of 1, a spike is emitted and v resets to 0.

4.2 Network Topology

- **Input layer:** 2048 neurons (`SpikeGeneratorGroup`, one per DVS pixel / polarity channel)
- **Hidden layer:** 128 recurrent LIF neurons (`NeuronGroup`)
- **No output layer** — classification is performed by a post-hoc logistic readout

The hidden layer is fully connected to itself via two competing synapse populations: sparse recurrent excitation sustains activity across time, while lateral inhibition creates winner-take-more competition between neurons, preventing the saturation seen in purely excitatory networks.

4.3 Hyperparameter Justification

Parameter	Value	Description
`width / height`	32×32	Sensor resolution (pixels)
`sim_duration_s`	0.4 s	Duration of each simulation trial
`fps`	200	Synthetic event emission rate
`bar_width`	4 px	Width of the moving bar
`base_speed`	100 px/s	Mean bar velocity across trials
`speed_std`	12 px/s	Trial-to-trial speed variability (Gaussian)
`temporal_jitter`	4 ms σ	Gaussian noise added to each event timestamp
`N_rec`	128	Number of hidden LIF neurons
`n_bins`	16	Time bins for feature extraction
`conn_p`	0.06	Input $\rightarrow$ hidden connection probability
`input_weight`	0.08	Synaptic weight: input $\rightarrow$ hidden (excitatory)
`recur_weight`	0.03	Synaptic weight: hidden $\rightarrow$ hidden (excitatory)
`inhib_weight`	0.05	Synaptic weight: hidden $\rightarrow$ hidden (inhibitory)
`tau_ms`	20 ms	LIF membrane time constant
`n_trials_train`	120 / cls	Training trials per class (240 total)
`clf_C`	1.0	Logistic regression inverse regularisation strength
`latency_n_spikes`	10	N-th spike threshold for latency measurement

5 Readout Classifier & Latency

5.1 Feature Extraction

After each trial, hidden spike activity is converted to a fixed-size feature vector:

- The 400 ms trial is divided into $n_\text{bins} = 16$ equal time bins (25 ms each)
- For each of the 128 hidden neurons, spikes are counted per bin
- Result: a $128 \times 16 = 2048$-dimensional feature vector per trial
- This preserves coarse temporal structure while keeping dimensionality tractable

5.2 Classifier Pipeline

The readout is a two-step `sklearn` Pipeline:

Step	Component	Purpose
1	MaxAbsScaler	Scales each feature to $[-1, 1]$ by dividing by its maximum absolute value. Crucially, zero-valued features remain zero — unlike `StandardScaler`, which centres features and turns zeros into non-zero values, causing latency bias artefacts.
2	LogisticRegression (C=1.0)	Linear classifier trained on the 2048-dim binned spike counts. Outputs a scalar decision function: positive = right, negative = left.

5.3 Latency Measurement

Latency is measured using the N-th spike method — a standard neuromorphic metric:

Definition: time at which the hidden population has fired a cumulative total of latency_n_spikes = 10 spikes since stimulus onset.

Why not use the classifier for latency: Feeding partial k-bin features into a pipeline trained on full 16-bin features is out-of-distribution. `MaxAbsScaler` and PCA produce non-zero transformations of the missing bins, creating class-specific artefacts (one class always deciding at the first allowed bin with std = 0 ms).

The N-th spike metric has no such dependency — it is pipeline-independent and directly measures the speed of network evidence accumulation.

6 Results

6.1 Summary Table

Metric	Value	Std	Notes
Train accuracy	100%	—	Synthetic task is linearly separable
Test accuracy	100%	—	Perfect generalisation on held-out 25%
Re-sim test accuracy	100%	—	Confirmed on freshly simulated test trials
Mean latency	14.7 ms	4.1 ms	Time of 10th hidden spike
Median latency	14.0 ms	—	Low median–mean gap → few outliers
Right class latency	10.8 ms	0.9 ms	Faster: bar enters from left edge immediately
Left class latency	18.6 ms	1.8 ms	Slightly slower: mirror trajectory
Total training trials	240	—	120 right + 120 left, balanced
Feature dimensionality	2048	—	128 neurons × 16 time bins

6.2 Classification Performance

The classifier achieves 100% accuracy on both the stored test set and freshly re-simulated trials. This is expected: the synthetic task is linearly separable by design, and the LIF network produces sufficiently distinct spike count patterns for the two motion directions. The consistency between stored-test and re-simulated accuracy confirms that the network generalises across trial-to-trial variability in speed and jitter.

6.3 Latency Analysis

The mean decision latency of 14.7 ms on a 400 ms trial (3.7% of the stimulus) demonstrates the ultra-low-latency inference characteristic of neuromorphic systems. Key observations:

- Rightward motion (10.8 ms mean) is consistently faster than leftward (18.6 ms). This is a genuine physical effect: a rightward bar enters from the left edge immediately, activating neurons with short-latency ON-polarity spikes. A leftward bar enters from the right edge and requires slightly more time for the random synaptic connectivity to converge on a stable population response.
- Low standard deviations (0.9 ms for right, 1.8 ms for left) confirm that the network responds consistently across trial-to-trial variation.
- The overall std of 4.1 ms is driven almost entirely by the between-class difference (8 ms gap) rather than within-class noise.

6.4 Simulation Outputs

Output File	Description
`event_maps.png`	ON and OFF DVS event heatmaps for a rightward bar
`rasters.png`	Input and hidden spike rasters, one trial per class
`confusion_matrices.png`	Stored-test and re-simulated test confusion matrices
`latency.png`	Latency histogram (all trials) + per-class breakdown
`spike_count_heatmaps.png`	Average spike count per neuron per bin, right vs. left
`membrane_voltages.png`	Voltage trace for neuron 0, one trial per class
`per_trial_metrics.csv`	Per-trial true label, prediction, correct flag, latency (ms)

7 Limitations & Future Work

7.1 Current Limitations

- **Synthetic sensor:** the DVS generator produces idealised events without realistic noise (hot pixels, refractory effects, bandwidth limits)
- **Fixed connectivity:** synaptic weights are hand-tuned constants; no learning rule (e.g. STDP) is used
- **Binary task:** only two classes (right / left). Real DVS datasets contain many more motion directions and object types
- **Small sensor:** 32×32 resolution; real DVS cameras operate at 346×260 or higher
- **No temporal coding:** the readout uses spike counts per bin, discarding precise spike timing

7.2 Suggested Next Steps

- **Real DVS data:** swap the synthetic generator for N-MNIST or DVS128 Gesture dataset loaders while keeping the rest of the pipeline unchanged
- **STDP learning:** replace fixed `input_weight` with Brian2's built-in spike-timing dependent plasticity to allow the network to self-organise
- **Multi-class extension:** add diagonal and circular motion classes; use a softmax readout
- **Speed:** parallelise trial simulation with `concurrent.futures.ProcessPoolExecutor` for near-linear CPU scaling
- **Temporal coding:** replace binned counts with Van Rossum distance or ISI-based features to exploit precise spike timing

8 Glossary

Term	**Definition**
DVS (Dynamic Vision Sensor)	Event-based camera that fires a spike for each pixel when luminance changes, producing asynchronous $(x, y, t, \text{polarity})$ events.
LIF (Leaky Integrate-and-Fire)	Simplified neuron model: membrane potential decays exponentially, spikes when threshold is crossed, resets immediately after.
SNN (Spiking Neural Network)	Neural network where information is encoded in the timing of discrete spikes rather than continuous activation values.
Polarity	Sign of luminance change: ON (polarity $= 0$) for brightness increase, OFF (polarity $= 1$) for decrease.
Lateral Inhibition	Inhibitory connections between neurons in the same layer; creates competition and sharpens selectivity.
STDP	Spike-Timing Dependent Plasticity — Hebbian learning rule where synaptic weight changes depend on the relative timing of pre- and post-synaptic spikes.
N-th Spike Latency	Time at which the population has fired N cumulative spikes; used here as a pipeline-independent latency metric.
Time-to-First-Spike (TTFS)	Special case of N-th spike latency with $N = 1$; often used in neuromorphic inference benchmarks.
Brian2	Open-source Python simulator for spiking neural networks with support for arbitrary neuron and synapse models.
Readout classifier	A simple linear (here logistic regression) classifier applied to hidden spike counts; does not modify the SNN itself.